Basics Day 3 — Session 3 (Hours 9–12)

Python Programming (Basic) • Comparisons, F-Strings, Text Processing & Debugging

Session 3 Overview

Topics Covered Today

- Hour 9: Comparisons + boolean logic — making decisions with code
- Hour 10: String formatting with f-strings — professional output
- Hour 11: Working with text — `split()` and `join()`
- Hour 12: Debugging habits — reading tracebacks, fixing bugs

Hour 9: Comparisons + Boolean Logic

Learning Outcomes

- Use all six comparison operators (`==` , `!=` , `<` , `>` , `<=` , `>=`)
- Explain the critical difference between `==` (comparison) and `=` (assignment)
- Combine conditions using `and` , `or` , and `not`
- Chain comparison operators in Python (Pythonic range checks)
- Apply boolean logic to build eligibility checks and decision rules

The Six Comparison Operators

```
age = 18

print(age == 18)    # True  — equality check
print(age == 21)    # False
print(age != 21)    # True  — not equal
print(age < 21)     # True  — less than
print(age > 21)     # False — greater than
print(age >= 18)    # True  — greater than or equal
print(age <= 18)    # True  — less than or equal
```

Every comparison produces exactly `True` or `False` — the only two boolean values.

= vs == — The Most Common Mistake

```
age = 18            # = assigns a value to a variable

if age == 18:       # == compares two values
    print("You are 18")
```

```
# ❌ WRONG — SyntaxError (assignment inside if)
if age = 18:
    print("You are 18")
```

> 💡 **Memory device:** `==` asks a question — "Are these equal?" Two equal signs, one question.

Real-World Comparisons

```python
# Password length check (uses len() from Day 2)
password = "secret"
min_length = 8
print(len(password) >= min_length)    # False — only 6 chars

# Case-sensitive string equality
user_answer = "Paris"
correct_answer = "paris"
print(user_answer == correct_answer)              # False
print(user_answer.lower() == correct_answer)      # True
```

> ⚠️ `"18"` (str) ≠ `18` (int) — always convert `input()` before comparing numbers

Chaining Comparisons (Pythonic)

```python
age = 25

# Standard (works in all languages)
if age >= 18 and age <= 65:
    print("Working age")

# Pythonic chained form — reads like math notation
if 18 <= age <= 65:
    print("Working age")   # same result, cleaner code
```

```python
score = 85
# Range check in one expression
print(70 <= score <= 100)   # True
```

Boolean Logic: `and` / `or` / `not`

```
age = 25
has_license = True
is_raining = False

# and — both must be True
print(age >= 18 and has_license)   # True

# or — at least one must be True
print(age >= 21 or has_license)    # True

# not — flips the value
print(not is_raining)              # True
```

Truth table summary:

`and` → both must be True

`or` → at least one must be True

`not` → reverses the boolean value

Combining Operators + Precedence

```python
age = 25
is_student = True
has_coupon = False

# Comparisons first, then not, then and, then or
if (age < 18 or is_student) and (not has_coupon):
    print("Discount applies")
```

> ⚠️ **Order matters:** `and` evaluates before `or`
>
> `a and b or c` means `(a and b) or c` — use parentheses to make intent explicit

Truthy and Falsy — A Preview

Falsy values (treated as `False` in boolean context):

```
""          # empty string
0           # zero
0.0         # zero float
None        # Python's null
False       # the boolean itself
```

```
# Practical example — check for empty input
age_input = input("Enter your age: ")

if age_input:                        # True if user typed something
    age = int(age_input)
    print(f"You are {age} years old")
else:
    print("You didn't enter anything!")
```

> 💡 For now, stick to **explicit** comparisons (`!= ""`, `> 0`) — easier to read

Demo: Membership Eligibility Checker

```python
# age_gate.py
print("=== Membership Eligibility Checker ===\n")

age = int(input("Enter your age: "))
has_membership = input("Do you have a membership? (yes/no): ").lower() == "yes"

if age >= 18 and has_membership:
    print("✓ Access granted! Welcome to the premium section.")
elif age >= 18 and not has_membership:
    print("✗ You are old enough, but you need a membership.")
elif age >= 13:
    print("✗ You must be 18 or older for this content.")
else:
    print("✗ This content requires adult supervision.")
```

Test boundary values: 12, 13, 17, 18, 65

Lab: Community Center Eligibility Checker

Time: 20–35 minutes | File: `eligibility_checker.py`

Rules to implement:

- **Children's Program:** age 5–12 (any residency)
- **Teen Program:** age 13–17 (any residency)
- **Adult Program:** age 18–64 AND local resident
- **Senior Program:** age 65+ (any residency)

```
age = int(input("Enter your age: "))
is_resident = input("Local resident? (yes/no): ").lower() == "yes"

if 5 <= age <= 12:
    print("✓ Children's Program")
elif 13 <= age <= 17:
    print("✓ Teen Program")
elif 18 <= age <= 64 and is_resident:
```

Lab Checkpoints (Hour 9)

Checkpoint 1 — Input & Conversion (5 min) ✓ Age converted to `int`, residency converted to `bool`

Checkpoint 2 — Core Logic (10 min) ✓ All four programs branch correctly ✓ Boundary values work: ages 12, 13, 18, 65

Checkpoint 3 — Debug & Verify (5 min) ✓ Non-resident adult case handled ✓ Code has comments on boundary checks

Common Pitfalls (Hour 9)

⚠️ `=` **instead of** `==` — SyntaxError inside `if` ⚠️ **String vs integer comparison** — `input()` returns a string; use `int()` ⚠️ **Off-by-one boundaries** — always test the exact threshold values ⚠️ `and` **before** `or` — use parentheses when mixing operators

```
# ❌ Dangerous ambiguity
if a and b or c:          # reads as (a and b) or c

# ✓ Explicit intent
if (a and b) or c:
```

Quick Check (Hour 9)

What will this print?

```
temperature = 75
is_sunny = True

if temperature >= 70 and is_sunny:
    print("Perfect beach day!")
else:
    print("Maybe another day")
```

Answer: `Perfect beach day!` — both conditions are `True`

Hour 10: String Formatting with F-Strings

Learning Outcomes

- Use f-strings to embed variables directly in strings (`f"{var}"`)
- Format floats to fixed decimal places (`{value:.2f}`)
- Apply width and alignment specifiers for columnar output
- Combine f-strings with `if/elif/else` for conditional messages
- Explain why f-strings beat concatenation and `str.format()`
- Avoid the four most common f-string beginner mistakes

The Problem: Messy Concatenation

```
bill = 87.5
tip_percent = 18
tip = bill * tip_percent / 100
total = bill + tip

# ❌ Old way — hard to read, manual str() conversion
print("Bill: $" + str(bill))
print("Tip (" + str(tip_percent) + "%): $" + str(tip))
print("Total: $" + str(total))
```

Output problems: `87.5` instead of `87.50`, hard-to-read code, error-prone.

F-String Basics

```python
name = "Alice"
age = 25

# Variables go right where they belong
print(f"My name is {name} and I am {age} years old")
```

```python
# Expressions work inside {} too
price = 19.99
quantity = 3
print(f"Total: ${price * quantity}")     # Total: $59.97

# Even method calls
name = "alice"
print(f"Hello, {name.upper()}!")         # Hello, ALICE!
```

> 💡 The `f` **prefix** before the quote is mandatory — it tells Python to evaluate `{}`

Format Specifiers: Numbers

```
pi = 3.14159265

print(f"2 decimals: {pi:.2f}")     # 3.14
print(f"4 decimals: {pi:.4f}")     # 3.1416
print(f"0 decimals: {pi:.0f}")     # 3
```

```
# Currency — always 2 decimal places
price = 19.5
tax   = 1.625
total = price + tax

print(f"Price: ${price:.2f}")      # Price: $19.50
print(f"Tax:   ${tax:.2f}")        # Tax:   $1.62
print(f"Total: ${total:.2f}")      # Total: $21.12
```

> 💡 `.2f` rounds to 2 decimal places — it does **not** truncate

Width and Alignment

```
# Left-align label (<), right-align number (>)
print(f"{'Item':<20} {'Price':>10}")
print(f"{'Coffee':<20} {'$3.50':>10}")
print(f"{'Sandwich':<20} {'$8.75':>10}")
```

```
Item                      Price
Coffee                    $3.50
Sandwich                  $8.75
```

```
# Combined: left label + right-aligned dollar amount
item = "Latte"
price = 4.5
print(f"{item:<15} ${price:>6.2f}")
# Latte           $  4.50
```

More Format Specifier Patterns

```
# GPA — one decimal
gpa = 3.8667
print(f"GPA: {gpa:.1f}")              # GPA: 3.9

# Percentage — manual calculation
correct, total = 17, 20
pct = correct / total * 100
print(f"Score: {pct:.1f}%")           # Score: 85.0%

# Large numbers with comma separator
population = 8336817
print(f"Population: {population:,}")  # Population: 8,336,817

# Currency + commas
revenue = 2493817.44
print(f"Revenue: ${revenue:,.2f}")    # Revenue: $2,493,817.44
```

F-Strings with Conditional Logic

```python
student_name = "Marcus"
score = 82.7

if score >= 90:
    print(f"{student_name} earned an A with {score:.1f}% — Excellent!")
elif score >= 80:
    print(f"{student_name} earned a B with {score:.1f}% — Good job!")
elif score >= 70:
    print(f"{student_name} earned a C with {score:.1f}% — Satisfactory")
else:
    print(f"{student_name} earned an F with {score:.1f}% — See instructor")
```

```
Marcus earned a B with 82.7% — Good job!
```

> 💡 `if/elif/else` picks the message; f-string makes it look good. A natural pair.

Four Common F-String Mistakes

```
# 1 — Space between f and quote (SyntaxError)
print(f "Hello, {name}!")    # ❌
print(f"Hello, {name}!")     # ✓

# 2 — Quote conflict (apostrophe ends string early)
# print(f'It's a {item}!')   # ❌
print(f"It's a {item}!")     # ✓ use double quotes

# 3 — Printing literal braces (use double braces)
print(f"{{name}} becomes {name}")  # ✓ → {name} becomes Alice

# 4 — Forgetting the f on one line (no error, wrong output!)
print(f"Item: {item}")
print("Quantity: {quantity}")   # ❌ prints literal {quantity}
```

Demo: Professional Tip Calculator

```python
# tip_calculator_formatted.py
bill = float(input("Enter bill amount: $"))
tip_percent = float(input("Enter tip %: "))

tip_amount = bill * (tip_percent / 100)
total = bill + tip_amount

print("\n--- Receipt ---")
print(f"{'Bill:':<15} ${bill:>8.2f}")
print(f"{'Tip (' + str(int(tip_percent)) + '%):':<15} ${tip_amount:>8.2f}")
print("-" * 26)
print(f"{'Total:':<15} ${total:>8.2f}")
print("\nThank you!")
```

```
--- Receipt ---
Bill:           $   87.50
Tip (20%):      $   17.50
--------------------------
Total:          $  105.00
```

Lab: Upgrade Tip Calculator

Time: 25–35 minutes | File: `tip_calculator_pro.py`

Requirements

- Prompt for bill amount and tip percentage
- Calculate tip and total
- Display a formatted receipt with:
  - All dollar amounts to **exactly 2 decimal places**
  - Clear labels, consistent alignment
  - A separator line before the total
  - A thank-you message
- Use **f-strings only** — no `+` concatenation

Completion Criteria

Common Pitfalls (Hour 10)

⚠️ **Forgetting the `f` prefix** — `"{name}"` prints literal text, no error ⚠️ **Space between `f` and quote** — `f "..."` is a SyntaxError ⚠️ **Apostrophe / quote conflict** — use double quotes when text has apostrophes ⚠️ **`{{` for literal braces** — `{` alone is always treated as a placeholder

```
item = "Coffee"
quantity = 2

print(f"Item: {item}")
print("Quantity: {quantity}")   # ❌ — forgot the f!
# Output: Quantity: {quantity}  ← silent bug
```

Quick Check (Hour 10)

What will this print?

```
name = "Alice"
score = 87.6789

print(f"{name} scored {score:.1f} points")
```

Answer: `Alice scored 87.7 points`

The `.1f` specifier rounds to 1 decimal place.

Hour 11: Working with Text — split() and join()

Learning Outcomes

- Use `.split()` to break a string into a list of words
- Understand how `.split()` handles whitespace and custom delimiters
- Use `.join()` to combine list elements into a single string
- Parse structured data (CSV-style) using `split(",")`
- Count words and find the longest word in a sentence
- Rebuild strings with custom separators
- Apply the split-process-join pattern

split() — String to List

```
sentence = "Python is awesome"

words = sentence.split()        # split on whitespace (default)
print(words)                    # ['Python', 'is', 'awesome']
print(type(words))              # <class 'list'>
print(len(words))               # 3
print(words[0])                 # 'Python'
print(words[-1])                # 'awesome'
```

`.split()` handles whitespace smartly:

```
text = "  Hello    world  "
print(text.split())             # ['Hello', 'world'] — strips & collapses

text = "one\ttwo\tthree"
print(text.split())             # ['one', 'two', 'three'] — tabs too
```

split() — Custom Delimiters

```
# CSV-style data
data = "John,Doe,25,Engineer"
fields = data.split(",")
print(fields)
# ['John', 'Doe', '25', 'Engineer']

# Phone number
phone = "555-123-4567"
parts = phone.split("-")
print(parts)    # ['555', '123', '4567']
```

Parsing and converting types together:

```
record = "Coffee Beans,12.99,3"
fields = record.split(",")

product = fields[0]
price = float(fields[1])
qty   = int(fields[2])
```

join() — List to String

```
words = ['apple', 'banana', 'cherry']

print(" ".join(words))       # apple banana cherry
print(",".join(words))       # apple,banana,cherry
print(", ".join(words))      # apple, banana, cherry
print("-".join(words))       # apple-banana-cherry
print(" | ".join(words))     # apple | banana | cherry
print("".join(words))        # applebananacherry
```

The syntax: separator comes FIRST

```
result = " and ".join(['Python', 'Java', 'JavaScript'])
print(result)    # Python and Java and JavaScript
```

> ⚠️ `.join()` requires a list of **strings** — use `str(n)` if you have numbers

Counting Words and Finding the Longest

```python
sentence = "Python is an amazing programming language"
words = sentence.split()

# Word count
print(f"Word count: {len(words)}")           # 6

# Longest word — beginner-friendly loop
longest = ""
for word in words:
    if len(word) > len(longest):
        longest = word

print(f"Longest word: '{longest}'")          # 'programming'
print(f"Its length: {len(longest)}")         # 11
```

The Round-Trip Relationship

`.split()` and `.join()` are **inverses** of each other:

```
# Clean messy whitespace in one line
messy = "  too   many    spaces   "
clean = " ".join(messy.split())
print(f"'{clean}'")
# 'too many spaces'
```

```
# Build a URL-friendly slug
title = "Python Programming Basics"
slug = "-".join(title.split()).lower()
print(slug)          # python-programming-basics
```

```
# Build a CSV line from variables
name, dept, score = "Alice", "Engineering", 92.5
line = ",".join([name, dept, str(score)])
print(line)          # Alice,Engineering,92.5
```

Demo: Sentence Statistics

```python
# sentence_stats.py
print("=== Sentence Statistics ===\n")

sentence = input("Enter a sentence: ")
words = sentence.split()

word_count = len(words)

longest = ""
for word in words:
    if len(word) > len(longest):
        longest = word

print(f"\nWord count: {word_count}")
print(f"Longest word: '{longest}' ({len(longest)} characters)")
print(f"Hyphenated: {'-'.join(words)}")
print(f"Piped: {' | '.join(words)}")
```

```
Enter a sentence: Python programming is incredibly powerful
```

Lab: Sentence Analyzer

Time: 25–35 minutes | File: `sentence_analyzer.py`

Requirements

1. Prompt the user for a sentence
2. Calculate and display:
   - Total word count
   - The longest word and its length
3. Show transformations:
   - Words joined with pipes: `word1 | word2 | word3`
   - Words joined with hyphens: `word1-word2-word3`
4. Use `.split()` and `.join()` appropriately

Example Output

Lab Completion Criteria (Hour 11)

✓ Accurate word count for any input ✓ Correctly identifies the longest word ✓ Both pipe and hyphen transformations are correct ✓ Uses `.split()` and `.join()` (no manual loops for joining) ✓ Clean formatted output using f-strings

Common Pitfalls (Hour 11)

⚠️ **`split()` returns a list** — not a string; use `len()` and indexing on it ⚠️ **Punctuation stays attached** — `"hello,"` is one word with the comma ⚠️ **Empty input** — `"".split()` returns `[]`; `len([])` is 0 ⚠️ **`join()` needs strings** — numbers must be converted first

```
# ❌ join() with integers fails
numbers = [1, 2, 3]
result = ", ".join(numbers)          # TypeError!

# ✓ Convert to strings first
result = ", ".join(str(n) for n in numbers)
print(result)    # 1, 2, 3
```

Quick Check (Hour 11)

What will this print?

```
items = ['Python', 'Java', 'JavaScript']
result = " and ".join(items)
print(result)
```

Answer: `Python and Java and JavaScript`

The separator `" and "` is inserted **between** each element.

Hour 12: Debugging Habits (Basics Level)

Learning Outcomes

- Read Python tracebacks from bottom to top
- Identify and fix the four most common error types (NameError, TypeError, ValueError, IndexError)
- Use `print()` strategically to check variable values and types
- Check your working directory when files are not found
- Form a hypothesis before making any code change
- Apply a systematic 6-step debugging process

What Is a Traceback?

```
Traceback (most recent call last):
  File "broken_script.py", line 7, in <module>
    result = calculate_average(score)
NameError: name 'score' is not defined
```

How to read it (bottom to top):

1. **Error type + message:** `NameError: name 'score' is not defined`
2. **Line number:** line 7
3. **Code at that line:** `result = calculate_average(score)`
4. **Root cause:** typo — variable is `scores` (with an 's')

> 💡 Tracebacks are **helpful diagnostic information** — not punishment.

Multi-Level Tracebacks

```
Traceback (most recent call last):
  File "multi_level.py", line 8, in <module>
    result = calculate_ratio(10, 0)
  File "multi_level.py", line 6, in calculate_ratio
    return divide_values(x, y)
  File "multi_level.py", line 3, in divide_values
    return a / b
ZeroDivisionError: division by zero
```

Reading the chain:

- Line 8 → called `calculate_ratio`
- Line 6 → called `divide_values`
- Line 3 → **error occurs here**

The **actual error** is always at the very bottom.

The Four Common Error Types

Error	Cause	Quick Fix
`NameError`	Variable/function doesn't exist	Check spelling; define before use
`TypeError`	Wrong type for operation	Convert with `str()`, `int()`, `float()`
`ValueError`	Right type, wrong value	Validate input before converting
`IndexError`	Index out of range	Indices start at 0; check `len()`

Error Examples: NameError and TypeError

```
# NameError — typo in variable name
name = "Alice"
print(nam)        # ❌ NameError: name 'nam' is not defined

# NameError — used before defined
print(greeting)   # ❌ NameError
greeting = "Hello"
```

```
# TypeError — can't add str + int
age = 25
message = "I am " + age              # ❌ TypeError
message = f"I am {age}"              # ✓ f-string fixes it

# TypeError — forgot to convert input()
bill = input("Enter bill: ")         # bill is a string
tip = bill * 0.18                    # ❌ TypeError
tip = float(bill) * 0.18             # ✓
```

Error Examples: ValueError and IndexError

```
# ValueError — right type, wrong value
age = int("twenty")    # ❌ ValueError: invalid literal for int()
age = int("20")        # ✓

# ValueError — wrong number of values to unpack
name = "Alice"
first, last = name.split()  # ❌ only one word
name = "Alice Johnson"
first, last = name.split()  # ✓
```

```
# IndexError — index out of range
fruits = ["apple", "banana"]
print(fruits[2])    # ❌ only indices 0 and 1 exist
print(fruits[1])    # ✓ "banana"

numbers = [1, 2, 3, 4, 5]
print(numbers[5])   # ❌ valid indices are 0–4
print(numbers[-1])  # ✓ last element
```

The 6-Step Debugging Process

1. **Reproduce** — run the script and observe the exact error
2. **Read** — read the traceback from bottom to top
3. **Hypothesize** — form a theory about the cause *before* changing anything
4. **Add print statements** — verify your hypothesis with `DEBUG` output
5. **Fix** — make the **smallest** change that addresses the root cause
6. **Re-run** — verify the fix works; repeat if needed

> 🔑 Change **one thing at a time**, then re-run. Never change multiple things at once.

Print-Debugging Strategies

Strategy 1 — Print variable values and types

```
bill = input("Enter bill: ")
tip_percent = input("Enter tip %: ")

# Add these to diagnose the crash
print(f"DEBUG: bill = {bill}, type = {type(bill)}")
print(f"DEBUG: tip_percent = {tip_percent}, type = {type(tip_percent)}")

tip = bill * (tip_percent / 100)  # ← crashes here
```

Output reveals: both are `str`, not `float` — add `float()` conversion.

Strategy 2 — Checkpoint prints

```
print("DEBUG: Starting program")
name = input("Enter name: ")
print("DEBUG: Got name")          # crash between these = this line failed
```

Print-Debugging: The DEBUG Flag Pattern

```python
DEBUG = True   # set to False when done

score1 = int(input("Enter score 1: "))
score2 = int(input("Enter score 2: "))
score3 = int(input("Enter score 3: "))

if DEBUG:
    print(f"DEBUG: score1={score1}, score2={score2}, score3={score3}")

total = score1 + score2 + score3

if DEBUG:
    print(f"DEBUG: total = {total}")

average = total / 3
print(f"Average: {average:.1f}")
```

One variable to turn all debug output on or off.

Checking Your Working Directory

```
FileNotFoundError: [Errno 2] No such file or directory: 'data.txt'
```

Python looks for files in the **current working directory**. If the file lives elsewhere, it won't be found.

```python
import os
print(f"Current directory: {os.getcwd()}")
```

Fix: Change to your project folder before running:

```
cd python_basics       # navigate to project folder
python my_script.py    # now Python looks here for files
```

> 💡 This is why we use **one course folder** — keeping everything together prevents path confusion

Demo: Systematic Break-and-Fix

```python
# buggy_calculator.py — score inputs not converted!
score1 = input("Enter score 1: ")
score2 = input("Enter score 2: ")
score3 = input("Enter score 3: ")

total = score1 + score2 + score3   # ❌ string concatenation
average = total / 3                 # TypeError: str / int

print(f"Average score: {average:.1f}")
```

Debug output reveals: `total = '859078'` (strings joined, not added)

Fix: `score1 = int(input("Enter score 1: "))`

After fix: correctly calculates `average = 84.3`

Lab: Debugging Drill

Time: 25–35 minutes | Fix three broken scripts

Script 1 — NameError (broken_greeter.py)

```
first_name = input("Enter your first name: ")
last_name = input("Enter your last name: ")
print(f"Hello, {first_nam} {last_name}!")   # typo!
```

Script 2 — TypeError (broken_temp_converter.py)

```
fahrenheit = input("Enter temperature in Fahrenheit: ")
celsius = (fahrenheit - 32) * 5 / 9    # str - int fails!
print(f"{fahrenheit}F is {celsius:.1f}C")
```

Script 3 — IndexError (broken_average.py)

```
numbers = [85, 92, 78, 90, 88]
fifth_number = numbers[5]    # only indices 0-4 exist!
```

Debugging Drill — For Each Script

1. Run it and observe the error
2. Read the traceback — what is the error type and line?
3. Add debug `print()` statements to confirm your hypothesis
4. Fix the bug (one change at a time)
5. Write **one sentence** explaining what went wrong

Expected Root Cause Explanations

- **Script 1:** `first_nam` is a typo — should be `first_name` (NameError)
- **Script 2:** `input()` returns a string — wrap with `float()` (TypeError)
- **Script 3:** List has 5 elements (indices 0–4) — use `numbers[4]` (IndexError)

Debugging Drill — Completion Criteria

✓ All 3 scripts run without errors ✓ Correct output verified with test inputs ✓ One-sentence root cause explanation written for each ✓ Debug `print()` statements removed from final code

Optional: Python 2 vs Python 3

> When you find old tutorials online — recognize these differences

Feature	Python 2	Python 3
print	`print "hello"` (statement)	`print("hello")` (function)
Division	`5 / 2 → 2` (integer)	`5 / 2 → 2.5` (float)
Input	`raw_input()`	`input()`
Strings	ASCII by default	Unicode by default

> 🔑 Always write Python 3. If you see `print "..."` online, it's Python 2 — translate the syntax.

Common Pitfalls (Hour 12)

⚠️ **Changing multiple things at once** — you won't know which fix worked ⚠️ **Not re-running after each fix** — always verify before moving on ⚠️ **Skipping the traceback** — it tells you exactly what and where ⚠️ **Leaving debug prints in final code** — clean them up before submission

Quick Check (Hour 12)

You see this error:

```
TypeError: can't multiply sequence by non-int of type 'float'
```

What's likely wrong and how do you fix it?

Answer: A string is being multiplied by a float — `input()` returns a string. Wrap the `input()` call with `float()` to convert it to a number first.

Session 3 Wrap-Up

What We Covered Today

Hour 9: Comparisons + Boolean Logic

- Six comparison operators; `==` vs `=` distinction
- `and`, `or`, `not` — combining conditions with correct precedence
- Chained comparisons: `18 <= age <= 65`
- Truthy/falsy preview; community center eligibility checker lab

Hour 10: F-Strings

- `f"{variable}"` syntax, expressions and method calls inside `{}`
- Number formatting: `:.2f`, `:.1f`, `:,`, `:,.2f`
- Width and alignment: `:<15`, `:>8.2f`

Scope Guardrail Reminder

Stay in Basics Scope ✓

- Comparison operators and boolean logic
- F-string formatting (`:.2f`, width, alignment)
- `split()` and `join()` for text processing
- Print-debugging and reading tracebacks

Not Yet — Advanced Topics ✗

- Regular expressions
- `try/except` error handling
- List comprehensions
- `str.format()` / `%`-style formatting
- Custom exception classes

Homework / Practice

1. **Grade Calculator** — use comparisons and `if/elif/else` to assign letter grades (A/B/C/D/F) from a numeric score; format output with f-strings
2. **Formatted Receipt** — build a multi-item receipt using f-string width/alignment so all prices line up in a right-justified column
3. **Word Frequency Counter** — split a sentence, count how many times each word appears, display a formatted count table
4. **Multi-Bug Script** — write a script with all four error types deliberately hidden; swap with a classmate and debug each other's code

Next Session Preview

Session 4 (Hours 13–16)

- Hour 13: Lists — create, index, slice, mutate
- Hour 14: Iterating lists with `for` loops — accumulators and averages
- Hour 15: Nested lists for table-like data — rows and columns
- Hour 16: Checkpoint 2 — Lists + string handling

Questions?

Key takeaways from today:

- `==` compares, `=` assigns — never confuse them
- F-strings: the `f` prefix is mandatory; `:.2f` for money; check every `{}`
- `.split()` → list of strings; `.join()` → string from list
- Debug systematically: read traceback → hypothesize → print → fix one thing → re-run

Thank You!

See you in Session 4!